实验接口与参数整理

本文档整理 `experiment/` 目录下“做实验”会用到的主要接口、配置参数、结果结构，以及和真实硬件相关的底层依赖。内容以当前统一实验框架 `exp_*` 为主，兼顾保留的历史兼容接口 `func_*` / legacy 脚本。

1. 推荐使用方式

当前推荐优先使用统一入口：

1. 快速启动：
   `result = exp_quickstart("mREV-GSS")`
2. 自定义配置后运行：
   `cfg = exp_default_config();`
   `result = exp_run_calibration(cfg);`
3. 兼容旧调用方式：
   `[calResult, result] = func_phasecali(s, v, cfg);`

建议优先级：

- 日常实验：`exp_quickstart`
- 需要精细控制参数：`exp_default_config` + `exp_run_calibration`
- 旧工程兼容：`func_phasecali`
- 非必要不建议直接使用 legacy 脚本
  `phase_calibration_golden_search_for_256ch_for_CCD.m`

2. 实验主流程

统一实验流程由 `exp_run_calibration` 串起，逻辑如下：

1. `exp_default_config` 生成默认配置 `cfg`
2. `exp_build_state(cfg)` 构造算法状态 `state`
3. `exp_get_algorithm_runner(cfg.method)` 选择算法执行器
4. `exp_make_channel_schedule(...)` 生成每轮通道访问顺序
5. 根据 `cfg.runtime.mockMeasureFn` 决定：
   - 非空：跳过真实硬件，直接使用 mock 测量函数
   - 为空：调用 `exp_init_hardware(cfg)` 初始化硬件，再通过 `exp_make_measure_fn(hw, cfg)` 生成真实测量回调
6. 执行算法 runner
7. `exp_close_hardware(hw)` 释放资源

3. 顶层可调用接口

3.1 `exp_quickstart(method)`

文件：`exp_quickstart.m`

用途：
快速以默认配置运行一次实验。

输入参数：

- `method`
  类型：文本标量
  可选值：`"mREV-GSS"`、`"5PPS"`、`"PFPD"`、`"CAIO"`
  默认值：`"mREV-GSS"`

输出：

- `result`
  类型：结构体
  内容：算法运行结果，字段见“结果结构”章节

适用场景：

- 想快速验证某种算法能否直接跑通
- 不想手动拼配置

3.2 `exp_default_config()`

文件：`exp_default_config.m`

用途：
返回统一实验配置结构体 `cfg`，是所有实验入口的配置基础。

输出：

- `cfg`
  类型：结构体
  内容：全局参数、测量参数、硬件参数、算法专属参数、运行时注入参数

3.3 `exp_run_calibration(cfg)`

文件：`exp_run_calibration.m`

用途：
统一实验入口，内部自动处理算法分发、通道调度、硬件初始化、测量函数生成与资源释放。

输入参数：

- `cfg`
  类型：结构体
  来源：通常由 `exp_default_config()` 生成后再修改

输出：

- `result`
  类型：结构体
  内容：最终控制量、评估次数、轮次结果等

说明：

- 若 `cfg.runtime.mockMeasureFn` 非空，则不访问真实硬件
- 若为空，则会初始化真实设备并在结束后自动清理

3.4 `func_phasecali(s, v, cfg)`

文件：`func_phasecali.m`

用途：
面向旧代码的兼容包装器，本质上还是调用 `exp_run_calibration(cfg)`。

输入参数：

- `s`
  类型：串口对象，可空
  含义：外部注入的电压源串口对象
- `v`
  类型：视频对象，可空
  含义：外部注入的 CCD 视频对象
- `cfg`
  类型：结构体，可空
  含义：实验配置，默认由 `exp_default_config()` 生成

输出：

- `cal_result`
  类型：数值向量
  含义：128 路有源通道对应的电压域结果 `sqrt(max(result.controlU, 0))`
- `result`
  类型：结构体
  含义：完整实验结果

适用场景：

- 旧代码已经持有串口/CCD 对象，不想改太多调用逻辑

4. 配置参数总览

`cfg` 的字段可以按以下类别理解。

4.1 全局算法控制参数

这些字段直接定义控制域和实验规模。

- `cfg.method`
  含义：选择算法
  可选值：`"mREV-GSS"`、`"5PPS"`、`"PFPD"`、`"CAIO"`

- `cfg.numChannels`
  含义：参与优化的通道数
  默认值：`128`
- `cfg.controlMin`
  含义：控制量下界，控制域为 `U`
  默认值：`0`
- `cfg.controlMax`
  含义：控制量上界，控制域为 `U`
  默认值：`7.5^2`
- `cfg.u2pi`
  含义：在 `U` 域中对应一个 `2π` 相移的跨度
  默认值：`0.036 * 1040`

核心约定：

- 当前统一框架的优化变量是 `U`
- 与电压关系为：`U = V^2`
- 仅在写硬件前才执行 `sqrt(U)` 转成电压 `V`

4.2 初始控制量

- `cfg.initialControlU`
  含义：初始控制向量
  类型：长度为 `cfg.numChannels` 的数值向量，或空数组
  默认值：`[]`

规则：

- 若为空，则在 `[controlMin, controlMax]` 内随机初始化
- 若非空，则会被 reshape 成行向量，并裁剪到 `[controlMin, controlMax]`

4.3 通道调度参数

字段位于 `cfg.channelOrder`。

- `cfg.channelOrder.mode`
  含义：每轮通道访问顺序生成方式
  可选值：
  - `"fixed"`：每轮都是 `1:numChannels`
  - `"random_each_round"`：每轮随机打乱
    默认值：`"random_each_round"`
- `cfg.channelOrder.seed`
  含义：随机种子
  默认值：`42`

说明：

- 实际调度矩阵由 `exp_make_channel_schedule(channelCfg, numChannels, rounds)` 生成
- 结果是大小为 `rounds x numChannels` 的矩阵

4.4 回滚参数

字段位于 `cfg.rollback`。

- `cfg.rollback.enableRoundRollback`
  含义：一轮结束后如果测量结果变差，是否回滚到本轮开始前的状态
  类型：逻辑值
  默认值：`true`

适用算法：

- `mREV-GSS`
- `5PPS`
- `PFPD`
- `CAIO`

4.5 测量参数

字段位于 `cfg.measure`。

- `cfg.measure.sampleTimes`
  含义：同一控制点下的重复采样次数
  默认值：`1`
- `cfg.measure.delaySec`
  含义：写电压后、采样前的等待时间，单位秒
  默认值：`0.03`
- `cfg.measure.ccdSpotSize`
  含义：CCD 积分区域的列宽
  默认值：`4`
- `cfg.measure.ccdCenterCol`
  含义：CCD 积分区域中心列
  默认值：`160`

说明：

- `CCD` 模式下会对整幅图像在列方向窗口内做积分
- `DSO` 模式下会读取波形并取平均电压
- `CAPTURE_CARD` 模式下会采集样本并取均值

4.6 硬件参数

字段位于 `cfg.hardware`。

4.6.1 通用字段

- `cfg.hardware.sensorMode`
  含义：选择测量设备
  可选值：`"CCD"`、`"DSO"`、`"CAPTURE_CARD"`
  默认值：`"CCD"`

- `cfg.hardware.serialPort`
  含义：256 路电压源串口号
  默认值：`"COM44"`
- `cfg.hardware.serialBaud`
  含义：串口波特率
  默认值：`128000`
- `cfg.hardware.useExternalSerial`
  含义：是否复用外部传入的串口对象
  默认值：`false`
- `cfg.hardware.useExternalVideo`
  含义：是否复用外部传入的 CCD 视频对象
  默认值：`false`
- `cfg.hardware.externalSerialObj`
  含义：外部串口对象
  默认值：`[]`
- `cfg.hardware.externalVideoObj`
  含义：外部视频对象
  默认值：`[]`

4.6.2 DSO 配置

字段位于 `cfg.hardware.dso`。

- `cfg.hardware.dso.vendor`
  含义：VISA 厂商
  默认值：`"NI"`
- `cfg.hardware.dso.resource`
  含义：示波器资源字符串
  默认值：`"USB0::0x0957::0x173B::MY54410002::0::INSTR"`
- `cfg.hardware.dso.channel`
  含义：读取波形的通道
  默认值：`"CHAN1"`
- `cfg.hardware.dso.points`
  含义：波形采样点数
  默认值：`250`

4.6.3 采集卡配置

字段位于 `cfg.hardware.capture`。

- `cfg.hardware.capture.dllDir`
  含义：`USB3000` 动态库目录
- `cfg.hardware.capture.headerDir`
  含义：`USB3000.h` 头文件目录
- `cfg.hardware.capture.sampleNum`
  含义：每次采样点数
  默认值：`1000`

- `cfg.hardware.capture.sampleTimes`
  含义：重复采样次数
  默认值：`1`
- `cfg.hardware.capture.sampleRateNs`
  含义：采样周期，单位 ns
  默认值：`1000`
- `cfg.hardware.capture.openChannels`
  含义：开启的输入通道数
  默认值：`1`
- `cfg.hardware.capture.sampleRange`
  含义：输入量程
  默认值：`1.28`

注意：

- `dllDir` 和 `headerDir` 是与本机驱动安装位置强绑定的硬编码路径
- 若换机器部署，需要优先核对这两个路径

4.7 运行时注入参数

字段位于 `cfg.runtime`。

- `cfg.runtime.mockMeasureFn`
  含义：mock 测量函数
  默认值：`[]`

说明：

- 非空时，`exp_run_calibration` 会跳过真实硬件初始化
- 适合无硬件联调、算法回归测试或快速验证接口行为

5. 算法专属配置参数

不同算法读取 `cfg` 中不同子结构。

5.1 `cfg.mrevGss`

对应 runner：`exp_run_mrevgss`

- `maxRounds`
  含义：总轮数
  默认值：`4`
- `kByRound`
  含义：每轮黄金分割搜索迭代深度参数
  默认值：`[7, 8, 10, 12]`
- `goldenRatio`
  含义：黄金比例系数
  默认值：`(sqrt(5)-1)/2`
- `controlMin`
  含义：控制域下界

- `controlMax`
  含义：控制域上界
- `channelSchedule`
  含义：可选，手动传入通道调度矩阵
- `enableRoundRollback`
  含义：可选，覆盖全局回滚策略

补充说明：

- 每轮容差 `gapTol = state.u2pi * (goldenRatio ^ kThisRound)`

5.2 `cfg.fivePps`

对应 runner：`exp_run_5pps`

- `maxRounds`
  含义：总轮数
  默认值：`4`
- `controlMin`
  含义：控制域下界
- `controlMax`
  含义：控制域上界
- `channelSchedule`
  含义：可选，手动传入通道调度矩阵
- `enableRoundRollback`
  含义：可选，覆盖全局回滚策略

补充说明：

- 单通道优化时固定采 5 个点
- 采样步长是 `deltaU = u2pi / 4`

5.3 `cfg.pfpd`

对应 runner：`exp_run_pfpd`

- `rounds`
  含义：总轮数
  默认值：`4`
- `numFitSamples`
  含义：每个通道拟合前采样点数
  默认值：`14`
- `fitGridPoints`
  含义：拟合完成后搜索峰值时的网格点数
  默认值：`201`
- `controlMin`
  含义：控制域下界
- `controlMax`
  含义：控制域上界

- `channelSchedule`
  含义：可选，手动传入通道调度矩阵
- `enableRoundRollback`
  含义：可选，覆盖全局回滚策略

补充说明：

- 内部使用 `fminsearch` 拟合余弦加线性趋势模型
- 若拟合失败，会回退到原始采样点中的最大值位置，并累计 `fitFailureCount`

5.4 `cfg.caio`

对应 runner：`exp_run_caio`

- `maxRounds`
  含义：总轮数
  默认值：`2`
- `ppsSteps`
  含义：单通道 PPS 采样点数
  可选值：`3`、`4`、`5`
  默认值：`5`
- `controlMin`
  含义：控制域下界
- `controlMax`
  含义：控制域上界
- `channelSchedule`
  含义：可选，手动传入通道调度矩阵
- `enableRoundRollback`
  含义：可选，覆盖全局回滚策略

补充说明：

- 每轮先更新奇数通道，再更新偶数通道
- 算法保留了交错优化和回滚逻辑，但不包含仿真中的显式串扰补偿

6. 运行时状态与中间接口

6.1 `exp_build_state(cfg)`

文件：`exp_build_state.m`

用途：
将 `cfg` 中算法运行所需的公共信息整理成 `state`。

输入：

- `cfg`

输出字段：

- `state.numChannels`
- `state.controlMin`

- `state.controlMax`
- `state.u2pi`
- `state.phasePerU`
  含义：`2*pi / u2pi`
- `state.initialControlU`

6.2 `exp_get_algorithm_runner(method)`

文件：`exp_get_algorithm_runner.m`

用途：
把方法名映射到具体 runner 函数句柄。

支持的 method token：

- `mREV-GSS` -> `@exp_run_mrevgss`
- `5PPS` / `FIVEPPS` -> `@exp_run_5pps`
- `PFPD` -> `@exp_run_pfpd`
- `CAIO` -> `@exp_run_caio`

说明：

- 内部会先转大写，再去掉非字母数字字符，因此如 `mREV-GSS`、`mrev gss` 之类写法都能归一化

6.3 `exp_make_channel_schedule(channelCfg, numChannels, rounds)`

文件：`exp_make_channel_schedule.m`

输入：

- `channelCfg`
  类型：结构体
- `numChannels`
  类型：正整数
- `rounds`
  类型：正整数

输出：

- `schedule`
  类型：`rounds x numChannels` 数值矩阵

7. 硬件初始化与释放接口

7.1 `exp_init_hardware(cfg)`

文件：`exp_init_hardware.m`

用途：
根据 `cfg.hardware.sensorMode` 初始化真实硬件资源。

输出：

- `hw`
  类型：结构体
  常见字段：
  - `sensorMode`
  - `serialObj`
  - `videoObj`
  - `dsoObj`
  - `capture`
  - `ownsSerial`
  - `ownsVideo`
  - `ownsDso`
  - `ownsCaptureLib`
  - `captureDeviceOpened`

三种模式下的行为：

- `CCD`
  - 串口始终会初始化或复用
  - CCD 会复用外部 `videoObj`，或内部创建 `videoinput("hamamatsu", 1, "MONO16_320x256")`
  - 内部设置：
    - `FramesPerTrigger = 1`
    - `ExposureTime = 0.10000e-03`
    - `TriggerSource = "internal"`
- `DSO`
  - 使用 legacy `instrfind` / `visa` 风格接口
  - 会设置波形源、采样点数、数据格式、字节序
- `CAPTURE_CARD`
  - 通过 `loadlibrary` 加载 `USB3000`
  - 打开设备、设置采样周期、模式、通道、量程、触发方式

7.2 `exp_close_hardware(hw)`

文件：`exp_close_hardware.m`

用途：
释放 `exp_init_hardware` 创建的资源。

释放原则：

- 仅释放由框架内部创建的对象
- 若对象由外部注入，则不会强制关闭

释放顺序：

1. CCD `videoObj`

2. DSO `dsoObj`
3. 采集卡设备句柄
4. `USB3000` 动态库
5. 串口对象 `serialObj`

8. 测量接口

8.1 `exp_make_measure_fn(hw, cfg)`

文件：`exp_make_measure_fn.m`

用途：
构造一个“给定控制向量，返回测量值”的回调函数。

输入：

- `hw`
  类型：硬件资源结构体
- `cfg`
  类型：配置结构体

输出：

- `measureFn`
  类型：函数句柄
  调用形式：`y = measureFn(controlU)`

输入约束：

- `controlU` 必须是长度为 `cfg.numChannels` 的数值向量
- 会被裁剪到 `[cfg.controlMin, cfg.controlMax]`

内部处理步骤：

1. `controlU -> voltage = sqrt(max(controlU, 0))`
2. `func_V_transition(voltage)` 生成 256 路电压向量
3. `ZJY_256VSRC_WRITE(hw.serialObj, voltageAllChannels)` 写入电压源
4. 按 `cfg.measure.delaySec` 等待
5. 根据 `sensorMode` 读取传感器

三种读数模式：

- `CCD`
  - 使用 `getsnapshot(hw.videoObj)` 抓图
  - 对中心列附近窗口做积分
  - 最终返回重复采样平均值
- `DSO`
  - 调用 `func_DSO_read(hw.dsoObj)`
  - 返回重复采样平均值
- `CAPTURE_CARD`

    - 调 `USB3GetAi`
    - 把所有采样拼接后取均值

8.2 `func_DSO_read(dsoObj)`

文件：`func_DSO_read.m`

用途：
从示波器读取一帧波形并返回平均电压。

输入：

- `dsoObj`
  含义：示波器对象

输出：

- `value`
  含义：波形均值

内部关键步骤：

1. 查询 `:WAVEFORM:PREAMBLE?`
2. 解析 `yIncrement`、`yOrigin`、`yReference`
3. 查询 `:WAVEFORM:DATA?`
4. 用 `binblockread(..., "uint16")` 读取原始数据
5. 转换为电压并求平均

9. 电压映射与设备写入接口

9.1 `func_V_transition(V_calibration)`

文件：`func_V_transition.m`

用途：
把 128 路有效通道电压映射成 256 路电压输出向量。

输入：

- `V_calibration`
  类型：数值向量
  长度：128

输出：

- `V`
  类型：长度为 256 的数值向量

当前实现特征：

- `V(129:256) = V_calibration(1:128)`
- 前 128 路目前保持 0

使用场景：

- `exp_make_measure_fn` 写硬件前必经该映射

风险提示：

- 这是非常关键的硬件通道映射点
- 若实验台通道接线或电压源通道定义变更，应优先检查此函数

9.2 `ZJY_256VSRC_WRITE(s, float_voltage_all_channels)`

文件：`ZJY_256VSRC_WRITE.m`

用途：
把 256 路电压向量编码后通过串口下发到 256 路电压源。

输入：

- `s`
  类型：串口对象
- `float_voltage_all_channels`
  类型：长度为 256 的行向量或列向量
  含义：目标电压

内部关键参数：

- `channels = 256`
- `bpc = 14`
  含义：DAC 位宽 14 bit
- `max_voltage = 12`
  含义：编码上限按 12V 处理

通信逻辑：

1. 电压值映射到 14 位整数
2. 每通道按十六进制拆分
3. 拼成 `VA ... Z` 指令字符串
4. 长字符串分包发送，每包最多约 510 个字符
5. 读取 ACK

ACK 校验：

- 最终回复必须包含 `ACK, ALL CHANNEL UPDATE`
- 否则抛出 `opa_exp:ZJY_256VSRC_WRITE:AckMismatch`

10. 各算法 runner 接口与结果字段

四个 runner 的函数签名一致：

`result = runner(state, cfgMethod, measureFn)`

其中：

- `state`：由 `exp_build_state` 构造
- `cfgMethod`：算法专属配置结构体
- `measureFn`：测量回调，输入 `controlU`，输出标量测量值

10.1 `exp_run_mrevgss(state, cfgMrevGss, measureFn)`

输出结果字段：

- `method = "mREV-GSS"`
  类型：字符串标量
  含义：结果对应的算法名称
- `controlU`
  类型：`1 x numChannels` 数值向量
  含义：算法结束时每个通道的最终控制量，处于 `U` 域，满足 `U = V^2`
- `evalCount`
  类型：标量整数
  含义：算法主流程内累计调用 `measureFn` 的次数，不含 `exp_run_calibration` 末尾那次可选复测
- `roundCount`
  类型：标量整数
  含义：实际运行轮数，这里等于 `cfgMrevGss.maxRounds`
- `initialIntensityMeasured`
  类型：标量数值
  含义：使用初始控制向量 `state.initialControlU` 测得的初始强度
- `roundIntensityMeasured`
  类型：`1 x roundCount` 数值向量
  含义：每轮结束后的轮级强度结果
  说明：若该轮触发回滚，则记录的是回滚后保留的强度值，也就是上一轮被接受的强度
- `roundGapTol`
  类型：`1 x roundCount` 数值向量
  含义：每轮黄金分割搜索使用的终止容差
  说明：第 `r` 轮按 `state.u2pi * (goldenRatio ^ kByRound(r))` 计算
- `roundMaxTerminalGap`
  类型：`1 x roundCount` 数值向量
  含义：每轮内所有通道单变量黄金搜索结束时的最大区间间隔
  说明：可用于粗略观察该轮搜索是否收敛到目标精度
- `roundEvalCount`
  类型：`1 x roundCount` 数值向量
  含义：每轮单独消耗的测量次数
  说明：包含该轮各通道搜索消耗的测量，以及轮末一次整体验证测量
- `roundAccepted`
  类型：`1 x roundCount` 逻辑向量
  含义：记录每轮结果是否被接受
  取值：
  - `true`：该轮更新被保留
  - `false`：该轮因强度下降而回滚

10.2 `exp_run_5pps(state, cfg5pps, measureFn)`

输出结果字段：

- `method = "5PPS"`
  类型：字符串标量
  含义：结果对应的算法名称
- `controlU`
  类型：`1 x numChannels` 数值向量
  含义：算法结束时每个通道的最终 `U` 域控制量
- `evalCount`
  类型：标量整数
  含义：算法主流程内累计测量次数
- `roundCount`
  类型：标量整数
  含义：实际运行轮数，这里等于 `cfg5pps.maxRounds`
- `initialIntensityMeasured`
  类型：标量数值
  含义：初始控制量对应的测量强度
- `roundIntensityMeasured`
  类型：`1 x roundCount` 数值向量
  含义：每轮 5 点相移法更新完成后的轮级强度
  说明：若发生回滚，则该轮记录回滚后保留的强度
- `roundEvalCount`
  类型：`1 x roundCount` 数值向量
  含义：每轮消耗的测量次数
  说明：理论上每个通道固定 5 次采样，再加轮末 1 次整体验证；若出现平坦响应提前保留原值，单通道仍已完成 5 次测量
- `roundAccepted`
  类型：`1 x roundCount` 逻辑向量
  含义：每轮结果是否通过回滚判定被接受

10.3 `exp_run_pfpd(state, cfgPfpd, measureFn)`

输出结果字段：

- `method = "PFPD"`
  类型：字符串标量
  含义：结果对应的算法名称
- `controlU`
  类型：`1 x numChannels` 数值向量
  含义：算法结束时的最终 `U` 域控制量
- `evalCount`
  类型：标量整数
  含义：算法主流程内累计测量次数

- `roundCount`
  类型：标量整数
  含义：实际运行轮数，这里等于 `cfgPfpd.rounds`
- `initialIntensityMeasured`
  类型：标量数值
  含义：初始控制向量下的测量强度
- `roundIntensityMeasured`
  类型：`1 x roundCount` 数值向量
  含义：每轮拟合峰值法更新完成后的轮级强度
- `roundEvalCount`
  类型：`1 x roundCount` 数值向量
  含义：每轮消耗的测量次数
  说明：主要由每通道 `numFitSamples` 次采样和轮末 1 次整体验证组成
- `fitFailureCount`
  类型：标量整数
  含义：整个运行过程中拟合失败的通道次数总和
  说明：拟合失败时算法会退回到该通道原始采样点中测量值最大的控制位置
- `roundAccepted`
  类型：`1 x roundCount` 逻辑向量
  含义：每轮结果是否被接受，若回滚则对应位置为 `false`

10.4 `exp_run_caio(state, cfgCaio, measureFn)`

输出结果字段：

- `method = "CAIO"`
  类型：字符串标量
  含义：结果对应的算法名称
- `controlU`
  类型：`1 x numChannels` 数值向量
  含义：奇偶交错优化完成后的最终 `U` 域控制量
- `evalCount`
  类型：标量整数
  含义：算法主流程内累计测量次数
- `roundCount`
  类型：标量整数
  含义：实际运行轮数，这里等于 `cfgCaio.maxRounds`
- `initialIntensityMeasured`
  类型：标量数值
  含义：初始控制量对应的测量强度
- `roundIntensityMeasured`
  类型：`1 x roundCount` 数值向量
  含义：每轮奇数通道、偶数通道交错更新完成后的轮级强度

- `roundEvalCount`
  类型：`1 x roundCount` 数值向量
  含义：每轮消耗的测量次数
  说明：单通道测量次数由 `cfgCaio.ppsSteps` 决定，可为 3、4 或 5，再加轮末 1 次整体验证
- `roundAccepted`
  类型：`1 x roundCount` 逻辑向量
  含义：每轮结果是否被接受，`false` 表示该轮更新被回滚

10.5 `exp_run_calibration` 额外补充的公共结果字段

无论使用哪种算法，`exp_run_calibration` 还会再补充这些字段：

- `initialControlU`
  含义：初始控制向量
- `methodSelected`
  含义：调用时选择的方法名

若使用真实硬件，还会补充：

- `finalIntensityMeasured`
  含义：最终控制量下再测一次的强度
- `evalCountWithFinal`
  含义：加上最终一次复测后的总评估次数

11. 历史兼容与辅助接口

11.1 `func_beam_steering(theta, s)`

文件：`func_beam_steering.m`

用途：
历史遗留的波束偏转辅助函数，不属于当前统一校准主流程。

输入：

- `theta`
  含义：目标偏转角
- `s`
  含义：串口对象

依赖：

- 当前目录存在 `Odu.txt`
- 当前目录存在 `d1.mat`

说明：

- 函数内部会给出 legacy warning
- 更适合历史流程复现，不建议作为日常标定/优化入口

11.2 `phase_calibration_golden_search_for_256ch_for_CCD.m`

文件：`phase_calibration_golden_search_for_256ch_for_CCD.m`

性质：

- 历史一体化脚本
- 集成了硬件初始化、采集、优化、清理等全部流程

当前状态：

- 默认被阻止执行
- 只有环境变量 `OPA_ALLOW_LEGACY_SCRIPT=1` 时才允许继续

不建议原因：

- 强依赖本机硬编码路径
- 依赖 legacy 仪器接口
- 可维护性和安全性都不如统一框架

11.3 `func_extract_FWHM(image)`

文件：`func_extract_FWHM.m`

用途：
从图像中提取相位方向和波长方向的半高全宽。

输入：

- `image`
  类型：二维图像矩阵

输出：

- `phase_FWHM`
- `wl_FWHM`

说明：

- 更偏实验结果分析，不直接参与统一校准主流程

11.4 `func_locate_wl_position(image)`

文件：`func_locate_wl_position.m`

用途：
定位图像中主峰所在的波长方向角位置。

输入：

- `image`

输出：

- `wl_position`

说明：

- 同样属于实验图像分析辅助函数

12. 外部依赖与注意事项

12.1 MATLAB/工具箱与设备依赖

当前代码涉及以下外部能力：

- `serialport`
- 图像采集接口 `videoinput` / `imaqreset` / `getsnapshot`
- legacy 仪器通信 `instrfind` / `visa`
- 动态库调用 `loadlibrary` / `calllib`
- 优化函数 `fminsearch`

12.2 当前目录中的数据/文件依赖

- `d1.mat`
  用途：legacy 波束偏转函数会读取变量 `d`
- `0du.txt`
  用途：`func_beam_steering` 需要
  现状：代码要求当前目录存在，但该文件不在当前目录清单中

12.3 最关键的参数检查点

每次正式实验前建议优先检查：

1. `cfg.hardware.sensorMode` 是否与当前设备一致
2. `cfg.hardware.serialPort` / `serialBaud` 是否正确
3. `cfg.hardware.capture.dllDir` / `headerDir` 是否匹配当前电脑
4. `cfg.numChannels` 与 `func_V_transition` 的硬件映射是否一致
5. `cfg.u2pi`、`controlMin`、`controlMax` 是否符合当前器件标定
6. `cfg.measure.delaySec`、`sampleTimes` 是否满足稳定时间需求

13. 常见修改入口

如果要做以下工作，优先改这些地方：

- 换算法：
  修改 `cfg.method`
- 改实验默认参数：
  修改 `exp_default_config.m`
- 改通道访问顺序：
  修改 `cfg.channelOrder` 或直接传入 `channelSchedule`
- 改电压到 256 路设备的映射：
  修改 `func_V_transition.m`

- 改测量定义：
  修改 `exp_make_measure_fn.m`
- 改硬件初始化方式：
  修改 `exp_init_hardware.m`
- 改旧接口兼容方式：
  修改 `func_phasecali.m`

14. 一页式索引

按用途快速查找：

- 快速跑实验：`exp_quickstart`
- 统一实验入口：`exp_run_calibration`
- 默认配置来源：`exp_default_config`
- 构造算法状态：`exp_build_state`
- 选择算法 runner：`exp_get_algorithm_runner`
- 生成通道顺序：`exp_make_channel_schedule`
- 初始化硬件：`exp_init_hardware`
- 关闭硬件：`exp_close_hardware`
- 生成测量回调：`exp_make_measure_fn`
- DSO 读数：`func_DSO_read`
- 电压映射到 256 路：`func_V_transition`
- 串口写 256 路电压源：`ZJY_256VSRC_WRITE`
- 旧兼容入口：`func_phasecali`
- 历史波束偏转：`func_beam_steering`
- 历史大脚本：`phase_calibration_golden_search_for_256ch_for_CCD.m`
- 图像分析辅助：`func_extract_FWHM`、`func_locate_wl_position`